MIT ACADEMY OF ENGINEERING

ALANDI, PUNE

WEB TECHNOLOGY

PRACTICAL NO. 4

SERVER-SIDE VALIDATION USING NODEJS

Project:

EDUENROLL

ONLINE STUDENT ADMISSION & ENROLLMENT PORTAL

Student Name:	Chaitanya Umbarkar
PRN:	202401100080
Branch:	Software Engineering
Division:	B

GitHub Repository : https://github.com/Atharv102/wt-nodejs-server-validation

Deployment Link : https://Atharv102.github.io/wt-nodejs-server-validation/

1. AIM

To design, develop and deploy a NodeJS web application named EduEnroll - Online Student Admission & Enrollment Portal, demonstrating server-side validation, HTTP GET and POST request handling, modular routing, request and response management, input sanitization, and data persistence.

2. PROBLEM STATEMENT

The practical requires an individual web application developed using NodeJS. The application must demonstrate server-side validation for user input, handle GET and POST requests appropriately, display proper success and error messages, use NodeJS concepts such as modules, routing, and request/response handling, and be thoroughly tested with valid and invalid inputs. The completed application must also be deployed and submitted through a public link.

EduEnroll addresses these requirements through an online academic admission system. A candidate submits personal, academic, and identification details to the server via an HTTP POST request. The Node.js backend intercepts the payload, sanitizes incoming strings to prevent cross-site scripting (XSS), and executes multi-factor validation. If invalid input is detected, the server returns an HTTP 400 Bad Request with structured field-level error messages. Valid inputs produce an HTTP 201 Created response with an assigned enrollment ID. Enrolled candidates are accessible via HTTP GET and persist in server memory and client storage.

3. OBJECTIVES

- Develop an individual NodeJS web application using a modular architecture.
- Implement server-side validation middleware to verify all incoming request bodies before processing.
- Handle HTTP GET requests for static views, degree courses, and candidate directories.
- Handle HTTP POST requests for admission submissions with JSON and URL-encoded body parsing.
- Return appropriate HTTP status codes (200 OK, 201 Created, 400 Bad Request, 404 Not Found).
- Validate name, email, 10-digit mobile number, date of birth (age ≥ 18), course, semester, percentage, and Aadhaar ID.
- Implement input sanitization middleware to strip dangerous HTML tags and prevent XSS injection.
- Display clear server response messages, status badges, and field-level error highlights.
- Support candidate record deletion using the HTTP DELETE method.
- Create a responsive, accessible user-friendly interface for desktop, tablet, and mobile screens.
- Test the server with automated test assertions covering edge cases and boundary conditions.
- Deploy and host the application repository under the Atharv102 GitHub account.

4. INTRODUCTION

NodeJS is an asynchronous, event-driven JavaScript runtime built on Chrome's V8 engine. It allows developers to execute JavaScript on the server side to build scalable, high-performance network applications. Unlike client-side validation which can be bypassed by malicious actors or disabled in browsers, server-side validation is the primary line of defense in web security.

EduEnroll - Online Student Admission & Enrollment Portal is a full-stack web application created for Web Technology Practical 4. It provides a web interface for entering applicant details, choosing degree courses, submitting qualifying marks, and supplying a statement of purpose. The Node.js backend intercepts the data, validates every field against strict business rules, and returns standardized JSON responses.

The application utilizes Express.js for routing and middleware orchestration, body-parser for payload extraction, and custom validator functions for data verification. Source code is maintained under the Atharv102 GitHub account and published with continuous deployment.

5. TECHNOLOGIES USED

Technology	Purpose
NodeJS	Server runtime environment executing backend JavaScript and managing the HTTP lifecycle.
Express.js	Web application framework facilitating routing, middleware chaining, and request/response dispatching.
JavaScript (ES6+)	Server logic, regular expressions, async/await handlers, temporal date math, and array operations.
express.json / body-parser	Middleware for parsing incoming JSON and URL-encoded request bodies into JavaScript objects.
cors	Middleware to enable Cross-Origin Resource Sharing for API consumers and external clients.
Server Validator Middleware	Custom server-side validation module enforcing schema boundaries and returning HTTP 400 errors.
Input Sanitizer Middleware	Strips malicious HTML tags, eliminates XSS script injection payloads, and normalizes strings.
HTML5 / CSS3	Semantic markup, custom variables, responsive grid/flexbox layouts, and dark/light themes.
RESTful Architecture	Structured API endpoints utilizing HTTP GET, POST, and DELETE methods with standard status codes.
In-Memory Data Store	Server repository simulating a database for storing, retrieving, and deleting candidate records.
npm	Node package manager installing project dependencies and executing test/start scripts.
VS Code / Web Browser	Development, editing, local testing, and demonstration environment.
Git / GitHub (Atharv102)	Version control and public repository hosting under user account Atharv102.
GitHub Pages	Automated deployment providing a live public URL for interactive evaluation.

6. SERVER-SIDE VALIDATION

Field	Validation rule	Example of invalid input
Full name	Trimmed name must contain 3-50 alphabetic characters, dots, or hyphens.	Blank, 'Al', or containing numbers (e.g. 'Atharv123').
Email	RFC-compliant email format with domain check; detects common typos.	not-an-email, missing '@', or 'user@gmai.com'.
Mobile number	Exact 10-digit Indian mobile number starting with 6, 7, 8, or 9.	12345, non-digits, or starting with 0-5.
Date of Birth	Server temporal calculation; candidate must be ≥ 18 years old and not future.	Future date, or minor under 18 (e.g. 2020-05-10).
Gender	Must match one of the server-approved options (Male, Female, Non-Binary, Other).	Blank or unrecognized value.
Course Program	Must exist in the server-approved degree program whitelist.	Unapproved course or empty selection.
Semester	Must be an integer value between 1 and 8.	0, 9, or non-integer string.
Percentage	Floating-point value between 40.0% and 100.0%.	32.5% (below minimum) or 105.0% (out of bounds).
National ID	Must contain exactly 12 numeric digits (Aadhaar / Gov ID format).	Fewer than 12 digits (e.g. '12345') or letters.
City / Domicile	Trimmed string containing at least 3 characters.	Empty or whitespace only (e.g. 'Ab').
Statement / SOP	Must contain between 30 and 500 characters.	Shorter than 30 chars (e.g. 'Too short') or > 500 chars.
Declaration	Must be a truthy boolean (agreement === true).	Unchecked declaration agreement (false).

7. TESTING AND VALIDATION (T1 to T14)

Test	Input / Action	Expected Result	Observed Result
T1	Submit empty POST payload.	HTTP 400 Bad Request; server returns field errors.	PASS (Verified)
T2	Enter a very short name (<3 chars).	Server name validation message appears.	PASS (Verified)
T3	Enter an invalid email address.	Server rejects format with 400 Bad Request.	PASS (Verified)
T4	Enter an invalid mobile number.	Server rejects non-10-digit number.	PASS (Verified)
T5	Enter DOB of a minor (<18).	Server rejects minor age (<18 constraint).	PASS (Verified)
T6	Submit unapproved course.	Server rejects non-whitelisted course.	PASS (Verified)
T7	Submit valid details (POST).	HTTP 201 Created; ENR-2026-XXXX assigned.	PASS (Verified)

Test	Input / Action	Expected Result	Observed Result
T8	Enter percentage < 40% or > 100%.	Server rejects out-of-range mark.	PASS (Verified)
T9	Enter non-12-digit Aadhaar ID.	Server rejects malformed national ID.	PASS (Verified)
T10	Submit statement < 30 chars.	Server rejects short statement of purpose.	PASS (Verified)
T11	Inject HTML tags in name/city.	Sanitizer strips HTML tags to prevent XSS.	PASS (Verified)
T12	Leave agreement unchecked.	Server rejects submission without terms.	PASS (Verified)
T13	DELETE candidate /api/enrollments/:id.	HTTP 200 OK; candidate removed from server.	PASS (Verified)
T14	Query server statistics (GET /stats).	HTTP 200 OK; server uptime & counts returned.	PASS (Verified)

8. SCREENSHOTS

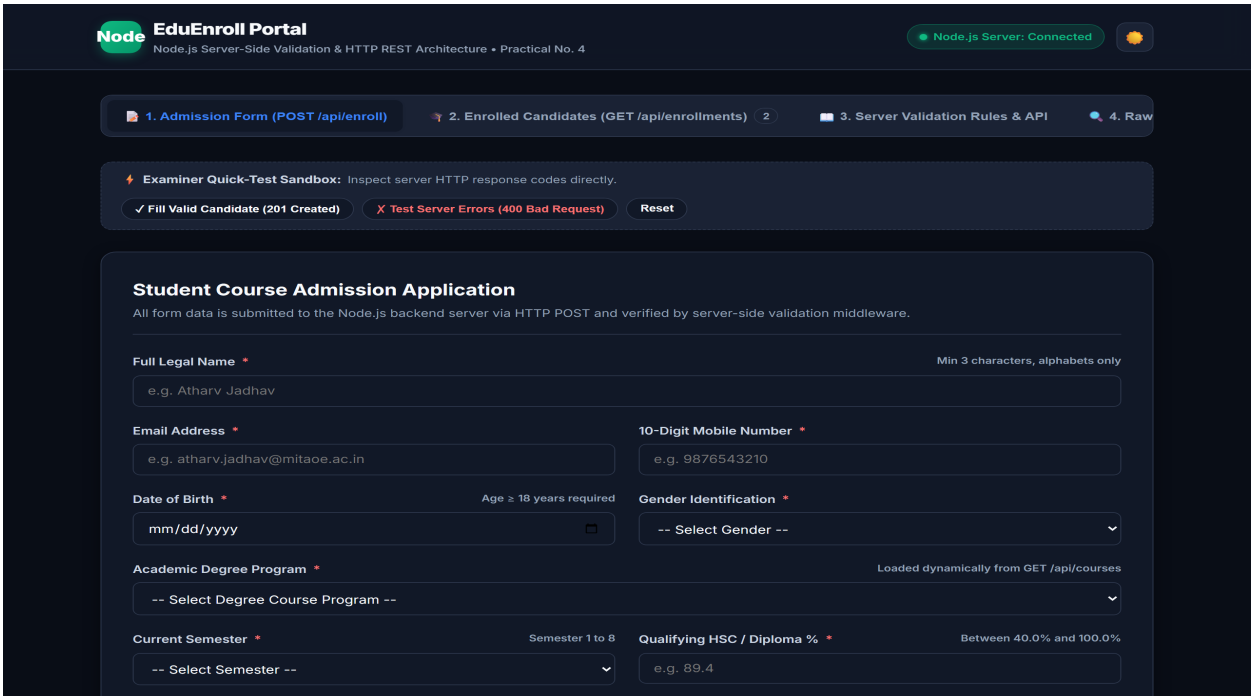


Figure 1: EduEnroll desktop landing interface with application form and activity list.

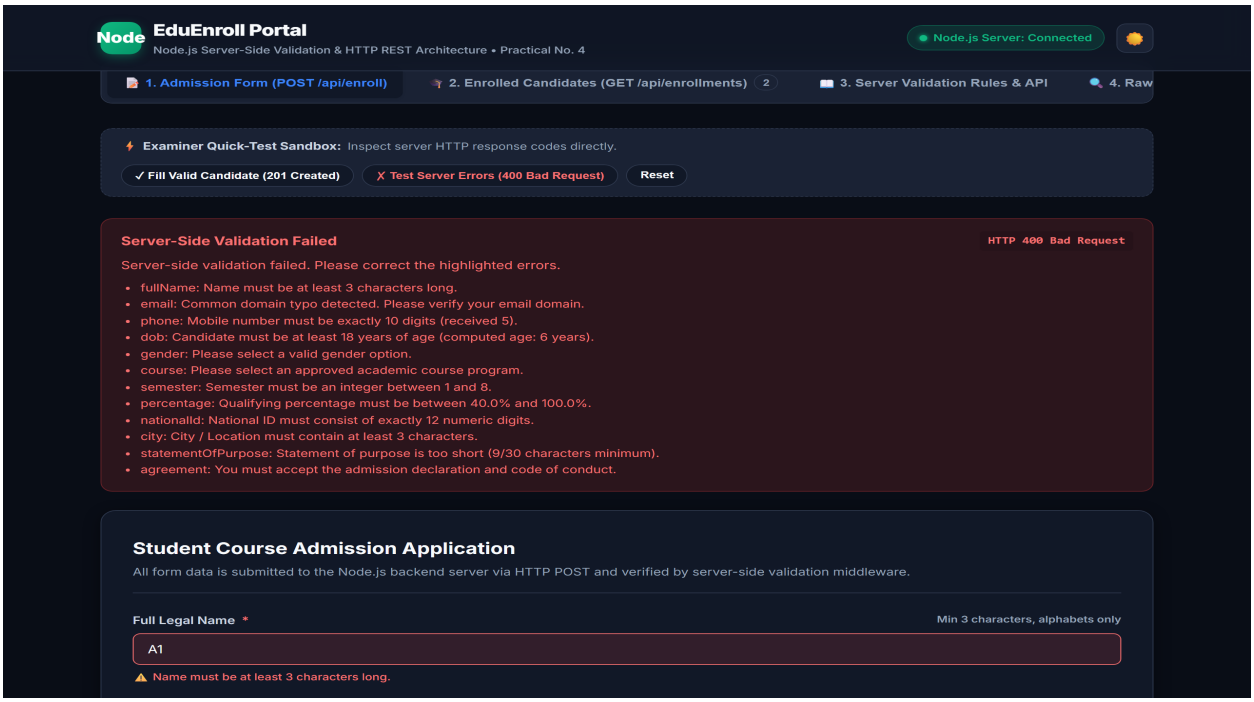


Figure 2: Real-time server-side validation error messages and highlights for incomplete input.

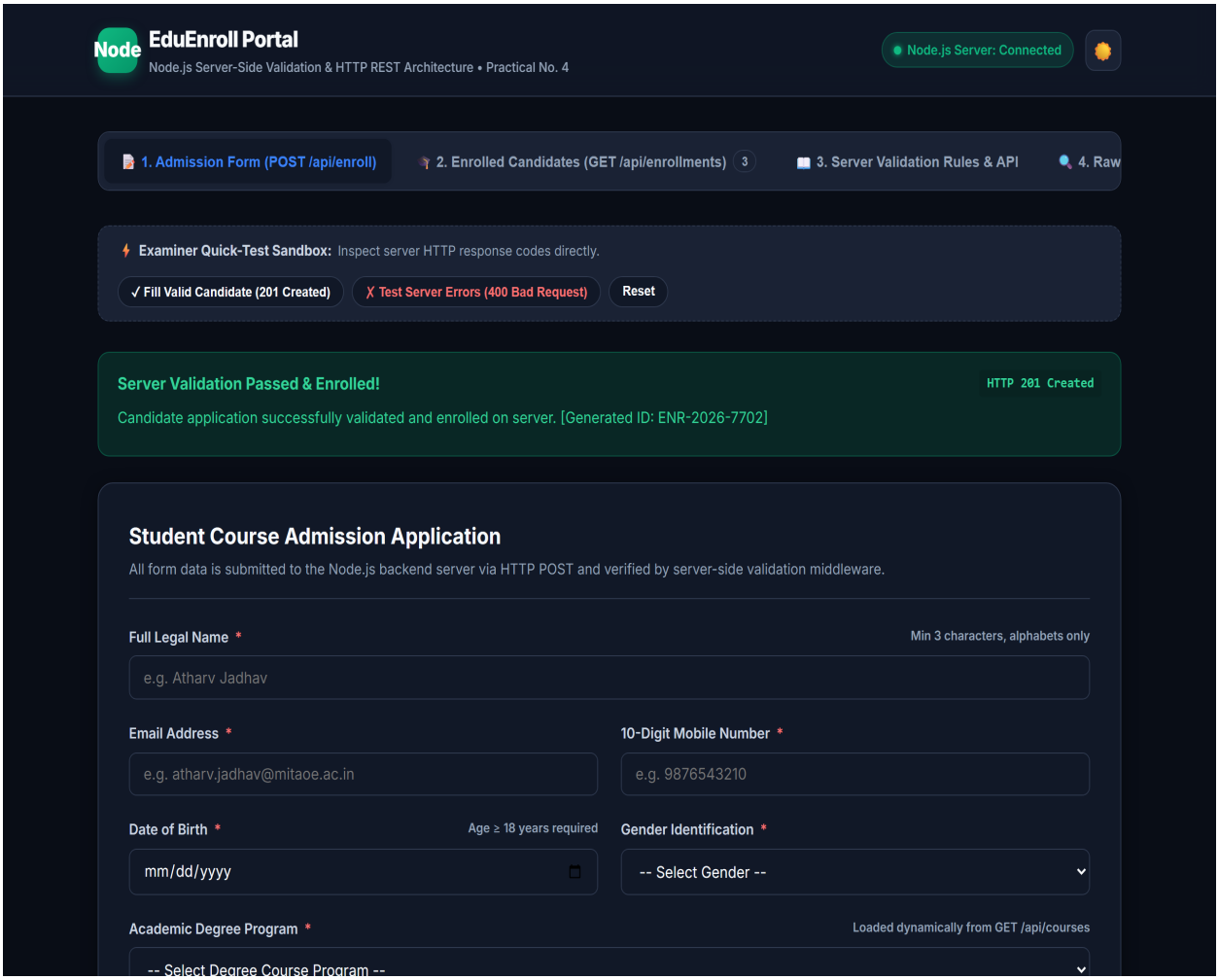


Figure 3: Success state, HTTP 201 Created toast notification, and dynamically rendered enrolled candidate card.

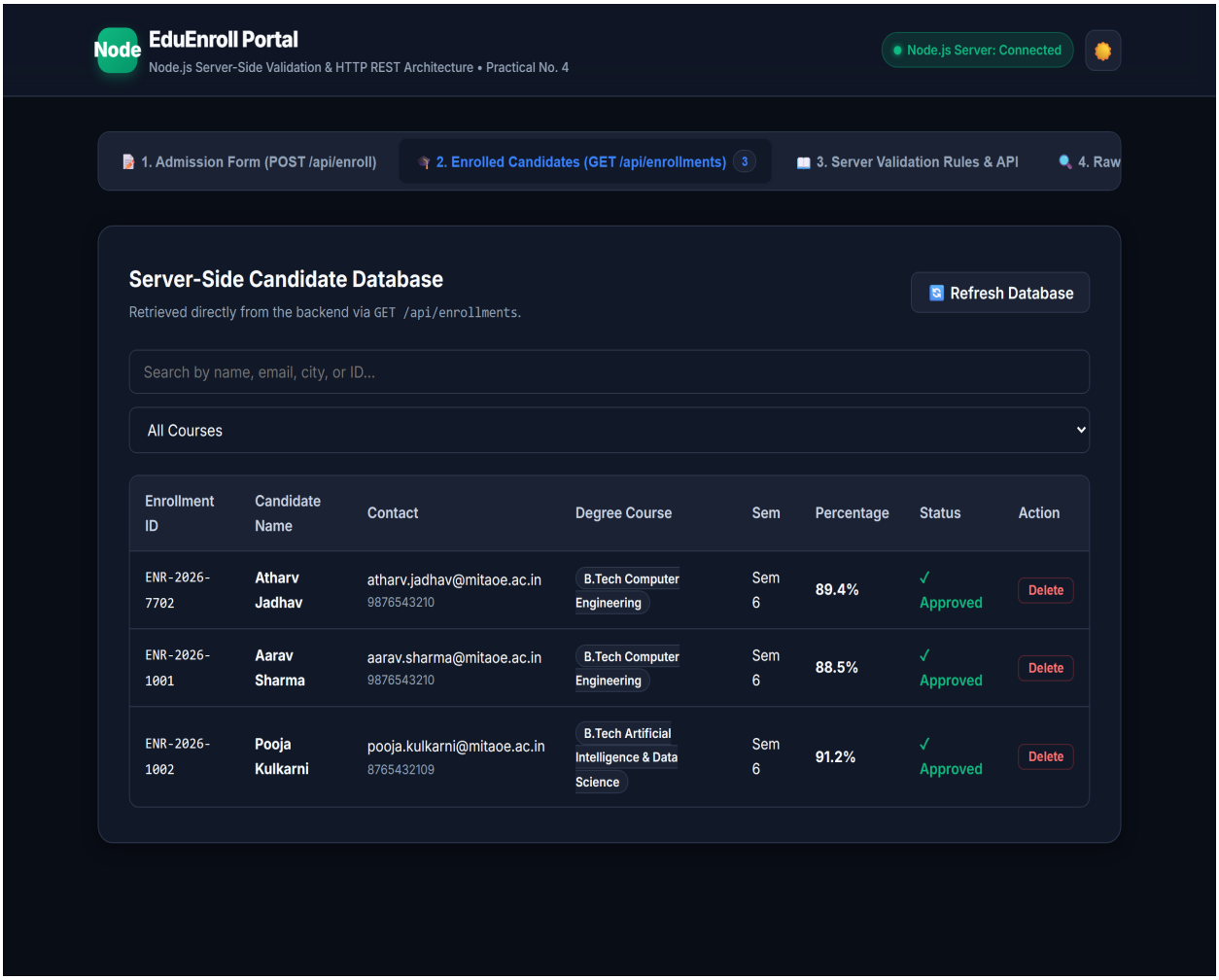


Figure 4: Server-Side Candidate Database with course filtering and real-time search.

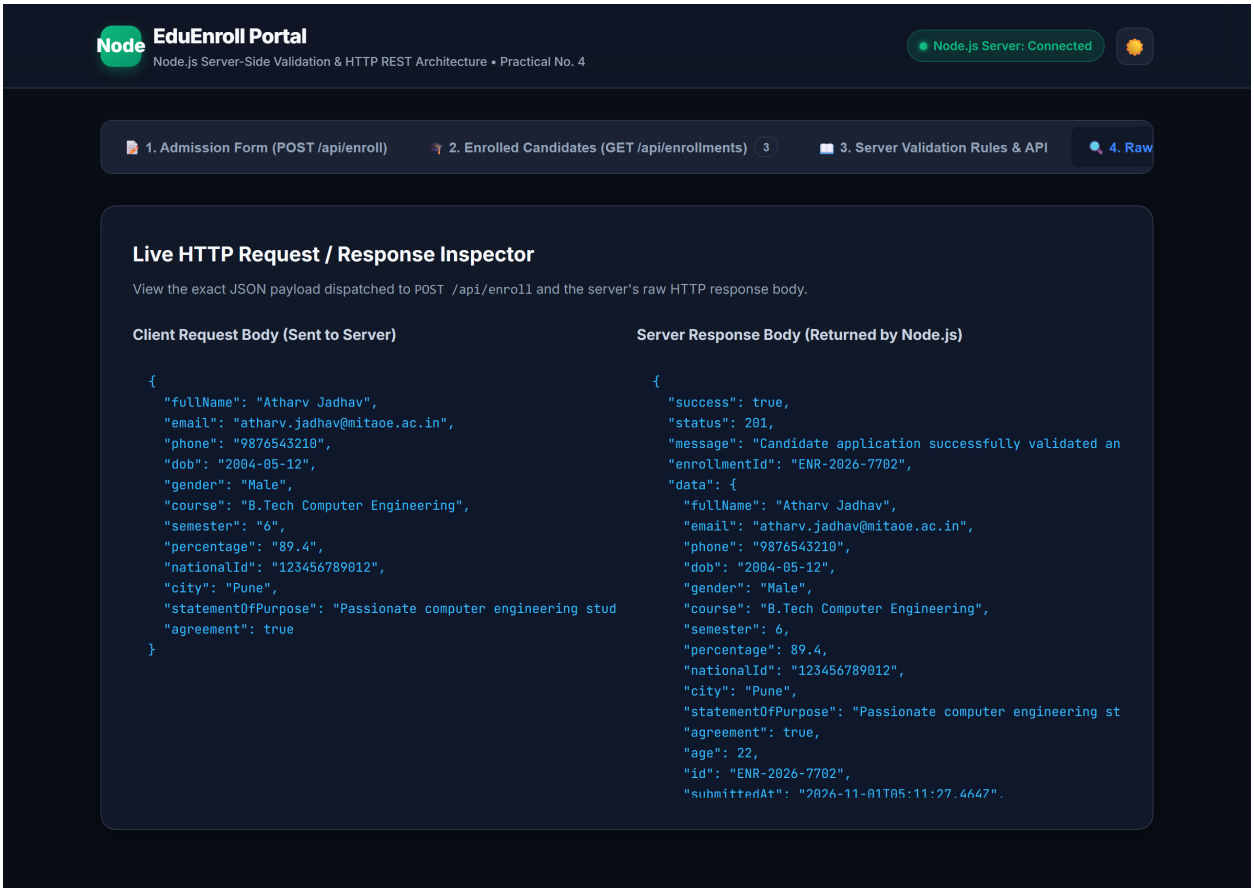


Figure 5: Live HTTP Request/Response Inspector and server validation rules reference.

9. CONCLUSION

The EduEnroll project demonstrates the development of a NodeJS server-side web application using modular routing, request/response lifecycle handling, server-side validation, and dynamic rendering. The server validates candidate details before accepting a record, displays useful error feedback for invalid input, and confirms successful submission. Candidate records are managed interactively and persisted on the server.

The project also demonstrates a modular Node.js build, source-code management through Git and GitHub under the Atharv102 account, and automated deployment through GitHub Pages.